

CODING CLUB

EPITECH

{EPITECH}
ACADEMY

Sujet : Fil d'Ariane

I. Introduction

En tant que mythologue, vous avez toujours été fasciné par le mythe du fil d'Ariane. Un beau jour, vous avez été envoyé pour résoudre le labyrinthe une bonne fois pour toutes.

Arrivé sur place, vous commencez à avancer dans le labyrinthe tout en posant des balises de repérage. Au bout de plus d'une heure, vous décidez de rebrousser chemin car vous n'avez plus de balise. En vous retournant, vous apercevez d'énormes traces de pied et toutes vos balises ont disparu.

Par chance, il vous reste votre téléphone. Servez-vous-en comme GPS pour atteindre la sortie. Aucune application n'existe pour cela : vous allez devoir la faire par vous-même !

Le but du projet est donc de résoudre un labyrinthe à l'aide d'un algorithme.

Pour cela, vous allez devoir coder en Python !



L'objectif est de faire un algorithme capable de vous sortir de n'importe quel labyrinthe. Il n'y a pas d'intérêt à écrire le chemin case par case, car il peut changer en fonction du labyrinthe.

Essayez de faire un algorithme simple pour commencer, puis, si vous vous sentez pousser des ailes, vous pourrez appliquer des algorithmes plus complexes que vous trouverez sur internet.

II. Trouver son chemin

Charger un labyrinthe

Avant tout, il vous faut un labyrinthe à résoudre. Chargez-en un avec l'une de ces fonctions, de la plus facile à la plus difficile :

```
print_map() # la plus facile
print_easy_map()
print_medium_map()
print_hard_map() # pour gagner la golden mark
```

Commencez par `print_map()` pour prendre vos marques, puis augmentez progressivement la difficulté à mesure que votre algorithme se perfectionne.

Droit dans le mur

Avant de commencer, vous pouvez tester le résultat attendu que vous allez devoir reproduire ; mais si vous préférez ne pas être spoilé, passez à l'étape suivante.

Si vous souhaitez tester quand même essayez ce code:

```
testVictory()
```

LabyTips & LabyTrics

Bon, cela étant, votre algo ne fait pas grand-chose pour le moment. Pour l'améliorer, vous devez créer l'algorithme.

Vous pouvez appeler les fonctions `right`, `down`, `left` et `up` pour vous diriger.

Avant de vous lancer dans votre algorithme, vous devez avoir en tête les points ci-dessous :

- Vous avez accès à une variable `map` qui contient toutes les valeurs à jour de la carte.
- Vous avez aussi accès aux variables `PosX` et `PosY` qui vous donnent votre position en les utilisant comme ceci : `map[PosY][PosX]` (ces variables sont mises à jour automatiquement).
- Les valeurs de la map sont :

caractère	définition
"x"	pour les murs
"_ "	pour les zones « libres »
"o"	pour la ligne d'arrivée
". "	pour les zones déjà visitées

- Le code est appelé une seule fois : à vous de vous arrêter à la fin du labyrinthe en utilisant la fonction `finish`.
- L'affichage est mis à jour à chaque fois que l'une des fonctions de mouvement est exécutée.
- Une variable `path` est mise à disposition. Vous connaîtrez son utilité dans la partie suivante.

En route vers la sortie

Pour bien commencer, essayez de vous déplacer d'une case à la fois pour comprendre le fonctionnement. Puis, ensuite, dans une direction jusqu'à un mur. Puis réfléchissez à la manière de changer de direction.

À un moment donné, vous serez dans un cul-de-sac. Auquel cas, vous devrez rebrousser chemin jusqu'à ce que vous puissiez changer de direction.

À vous de jouer !

III. Le bout du tunnel

Félicitations, vous avez un algorithme fonctionnel. Vous allez pouvoir partir à l'aventure !

Maintenant que vous avez votre GPS pour labyrinthe, vous allez pouvoir continuer de l'améliorer.

- Essayez d'optimiser votre algorithme pour qu'il soit plus rapide.

Si vous êtes curieux, pensez à poser vos questions aux Mantas. Ils seront ravis de partager leurs connaissances avec vous.